

UNIT 3

❖ What is Dynamic Testing?

[Dynamic Testing](#) is a type of [Software Testing](#) that is performed to analyze the dynamic behavior of the code. It includes the testing of the software for the input values and output values that are analyzed. It is performed to describe the dynamic behavior of code. It refers to the observation of the physical response from the system to variables that are not constant and change with time.

Objectives of Dynamic Testing

1. **Find errors and bugs:** Through comprehensive testing, find and expose flaws, faults, or defects in the software code and its functionality so that they can be fixed as soon as possible.
2. **Verify the behavior of the system:** Verify that the software operates as expected and complies with company requirements, industry or regulatory standards, user expectations, and any applicable business regulations.
3. **Assessing Performance:** To make sure the software satisfies performance requirements, evaluate its performance by monitoring reaction times, throughput, and use of resources under various scenarios.
4. **Assure Trustworthiness:** Examine the software's dependability by determining how well it performs regularly under typical operating conditions, free of unexpected faults or crashes.
5. **Accuracy of Test Data:** Verify the precision and consistency of the data handled by the software to guarantee reliable and uniform information handling.
6. **Assess Scalability:** Examine whether the application can grow to handle more users, workloads, or data volumes without seeing an obvious decline in performance.

Levels of Dynamic Testing

Several levels of dynamic testing are commonly used in the software development process, including:

1. **Unit testing:** Unit testing is the process of testing individual software components or “units” of code to ensure that they are working as intended. Unit tests are typically small and focus on testing a specific feature or behavior of the software.
2. **Integration testing:** Integration testing is the process of testing how different components of the software work together. This level of testing typically involves testing the interactions between different units of code, and how they function when integrated into the overall system.

3. **System testing:** System testing is the process of testing the entire software system to ensure that it meets the specified requirements and is working as intended. This level of testing typically involves testing the software's functionality, performance, and usability.
4. **Acceptance testing:** Acceptance testing is the final stage of dynamic testing, which is done to ensure that the software meets the needs of the end-users and is ready for release. This level of testing typically involves testing the software's functionality and usability from the perspective of the end-user.
5. **Performance testing:** Performance testing is a type of dynamic testing that is focused on evaluating the performance of a software system under a specific workload. This can include testing how the system behaves under heavy loads, how it handles a large number of users, and how it responds to different inputs and conditions.
6. **Security testing:** Security testing is a type of dynamic testing that is focused on identifying and evaluating the security risks associated with a software system. This can include testing how the system responds to different types of security threats, such as hacking attempts, and evaluating the effectiveness of the system's security features.

❖ Black Box Testing

BBT-It defines the functionality of the application/software/build as per requirement.

There are 2 main types → 1. Functionality Testing 2. Non Functionality Testing

1. Functionality Testing → In the functionality testing we validate the functionality of the build/application with respect to their internal feature Ex- Paytm → Recharge functionality testing
2. Non Functionality Testing → In the non-functionality testing we validate the functionality of the build/application with respect to their external feature Ex-Paytm application open in different browsers like on chrome, MS edge etc..

Functionality Testing →

In the functionality testing we validate the functionality of the build/application with respect to their internal feature. There are 6 types of functionality testing (**BIEBSC**)

1. Behavioral coverage Testing
2. Input domain coverage Testing
3. Error handling coverage Testing
4. Backend coverage/Data base Testing
5. Service level coverage Testing

6. Calculation base coverage Testing

1. . Behavioral coverage Testing→

- In this testing as tester we validates the behavior of the object (web element)
- Validates the object property & behavior

2. Input domain Coverage Testing→

In input domain coverage testing as tester we are validating the input which we are passing ion to the object (web element)

In that checking→Input data type, size & length of the input

We maintain these into

BVA (Boundary Value Analysis)→ In BVA we maintained the size & length of the input which we are passing in to the object (web element).

3. . Error handling coverage Testing→

As tester we are validating the different types of error message present in application/build. If we will provide invalid data then standard error message should be display

4. . Backend coverage/Data base Testing→

- In the data base testing we are validating the whatever activities we are doing that stored in data base or not
- By using SQL server queries we will validates the changes in data base as per the front end

5. .Service Level coverage Testing→

As tester we validates the sequential flow of the application is following

Ex-Paytm recharge module→Recharge module→Mobile No→Circle selection→Plan selection→payment→Success message

6. Calculation base coverage Testing →

In calculation base coverage testing we validate the arithmetic calculation in the application as per the requirement

Non Functionality Testing→

Here as tester we are validating the external feature of the application/build

There are different types of non-functionality testing→

1. Recovery Testing
 2. Configuration Testing
 3. Compatibility Testing
 4. Installation Testing
 5. Parallelization Testing
 6. Sanitization Testing
 7. Globalization Testing
 8. Intersystem Testing
- } Not performed

❖ ECP (Equivalent Class Partition)→

In ECP we are validating the data type of input which we are passing in to the object (web element) and we divide the limit in particular partition so by applying this approach in test case design we can decrease the test case count and test the functionality smart way.

Ex-Login page for Facebook for user name

Pass→Mobile number as int, fail→char

Object box	pass	fail
Mobile no	10 digit	< 10 digit or > 10 digit
Mail id	String	Char/int
Mail id	Symbol, int	Black/null

❖ **Boundary Value Analysis**

[Boundary Value Analysis](#) is based on testing the boundary values of valid and invalid partitions. The behavior at the edge of the equivalence partition is more likely to be incorrect than the behavior within the partition, so boundaries are an area where testing is likely to yield defects.

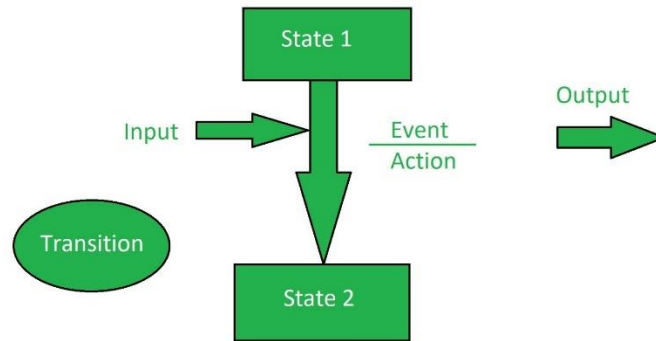
It checks for the input values near the boundary that have a higher chance of error. Every partition has its maximum and minimum values and these maximum and minimum values are the boundary values of a partition.

Note:

- A boundary value for a valid partition is a valid boundary value.
- A boundary value for an invalid partition is an invalid boundary value.
- For each variable we check-
 - Minimum value.
 - Just above the minimum.
 - Nominal Value.
 - Just below Max value.
 - Max value.

❖ **State Transition Testing**

is a type of software testing which is performed to check the change in the state of the application under varying input. The condition of input passed is changed and the change in state is observed. State Transition Testing is basically a black box testing technique that is carried out to observe the behavior of the system or application for different input conditions passed in a sequence. In this type of testing, both positive and negative input values are provided and the behavior of the system is observed. State Transition Testing is basically used where different system transitions are needed to be tested.



Objectives of State Transition Testing

The objective of State Transition testing is:

- **Determining the System States:** Identify and record the different states that the system that is being tested can be in. These phases are frequently connected to certain circumstances or occurrences inside the system.
- **State Transition Modelling:** It makes a model or state transition diagram that shows the system's various states and the changes between them.
- **Verifying Legitimate State Transitions:** Check that the system responds to various inputs or events by switching between valid states in a correct manner. By doing this, it is ensured that the system transitions between states with expected behaviour.
- **Checking the Beginning and End States:** Verify that, following a series of actions or events, the system finishes in the expected final state and begins in the proper beginning state. This guarantees correct system setup and dismantling.
- **Error Correction and Recovery:** Examine the system's capacity to recover from unforeseen conditions and accept faults with patience. This covers situations in which the system operates and runs into unexpected inputs or events.

❖ Cause Effect Graphing

Cause Effect Graphing based technique is a technique in which a graph is used to represent the situations of combinations of input conditions. The graph is then converted to a decision table to obtain the test cases. Cause-effect graphing technique is used because boundary value analysis and equivalence class partitioning methods do not consider the combinations of input conditions. But since there may be some critical

behaviour to be tested when some combinations of input conditions are considered, that is why cause-effect graphing technique is used.

Basic Notations used in Cause-effect graph:

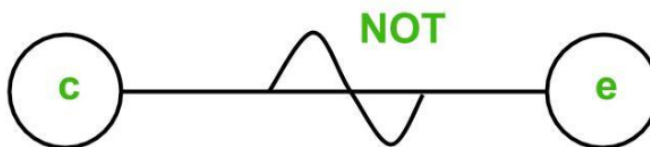
Here c represents cause and e represents effect.

The following notations are always used between a cause and an effect:

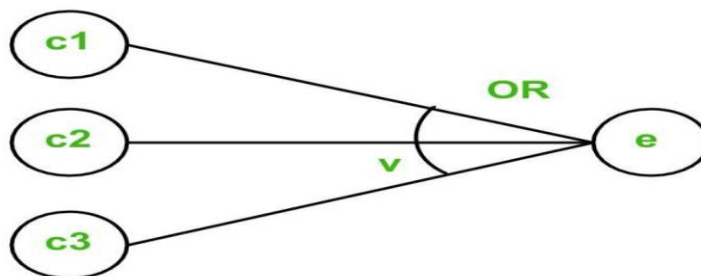
1. Identity Function: if c is 1, then e is 1. Else e is 0.



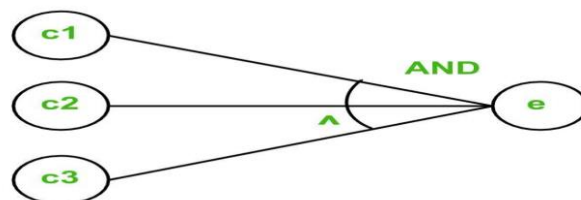
2. NOT Function: if c is 1, then e is 0. Else e is 1



3. OR Function: if c1 or c2 or c3 is 1, then e is 1. Else e is 0



4. AND Function: if both c1 and c2 and c3 is 1, then e is 1. Else e is 0.



❖ Decision table technique

Decision table technique is one of the widely used case design techniques for black box testing. This is a systematic approach where various input combinations and their respective system behavior are captured in a tabular form.

That's why it is also known as a cause-effect table. This technique is used to pick the test cases in a systematic manner; it saves the testing time and gives good coverage to the testing area of the software application.

Decision table technique is appropriate for the functions that have a logical relationship between two and more than two inputs.

❖ What is Use Case Testing?

Use Case Testing is generally a part of black box testing and that helps developers and testers to identify test scenarios that exercise the whole system on each transaction basis from start to finish. Business experts and developers must have a mutual understanding of the requirement, as it's very difficult to attain.

Feature of Use Case Testing

- **User-focused Approach:** The interactions and scenarios that users have with the system are the main focus of testing. It places a strong emphasis on verifying the system's response to user input and activities.
- **Scenario-Based Testing:** Use case scenarios, which depict common or important interactions between users and the system, serve as the framework for organizing testing. This strategy helps in making sure that the system lives up to user expectations in practical settings.
- **Requirements Validation:** Use Case Testing verifies that the system satisfies the specifications given in the use cases. Testing makes ensuring that the functional requirements are implemented correctly, as each use case usually reflects a particular functional requirement.

❖ White Box Testing Statement Coverage

Statement coverage is one of the widely used software testing. It comes under white box testing.

Statement coverage technique is used to design white box test cases. This technique involves execution of all statements of the source code at least once. It is used to calculate the total number of executed statements in the source code out of total statements present in the source code.

Statement coverage derives scenario of test cases under the white box testing process which is based upon the structure of the code.

In white box testing, concentration of the tester is on the working of internal source code and flow chart or flow graph of the code.

Generally, in the internal source code, there is a wide variety of elements like operators, methods, arrays, looping, control statements, exception handlers, etc. Based on the input given to the program, some code statements are executed and some may not be executed. The goal of statement coverage technique is to cover all the possible executing statements and path lines in the code.

❖ Branch Coverage Testing

Branch coverage technique is used to cover all branches of the control flow graph. It covers all the possible outcomes (true and false) of each condition of decision point at least once. Branch coverage technique is a whitebox testing technique that ensures that every branch of each decision point must be executed.

However, branch coverage technique and decision coverage technique are very similar, but there is a key difference between the two. Decision coverage technique covers all branches of each decision point whereas branch testing covers all branches of every decision point of the code.

❖ Branch Coverage

Branch coverage in unit testing is a metric that measures the percentage of branches (decision points) in the source code that have been executed during the testing process. It indicates how well the test cases navigate through different possible outcomes of conditional statements, helping evaluate the thoroughness of testing. A high branch coverage means that most decision paths in the code have been tested, increasing the likelihood of detecting potential defects.

Formula for Branch Coverage:

$$\text{Branch Coverage} = (\text{Total Number of Branches}) \div (\text{Number of Executed Branches}) \times 100\%$$

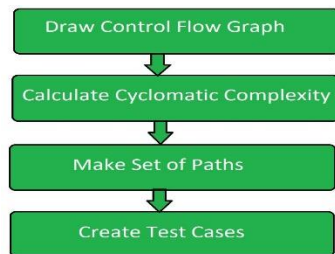
❖ Purpose of Branch Coverage

1. **Comprehensive Test Coverage:** The primary purpose of Branch Coverage in unit testing is to ensure test coverage by targeting each branch statement within code blocks or functions.
2. **Conditional Statements Coverage:** Branch Coverage addresses the coverage of conditional statements such as if, if...else, while, for, and do...while statements in the source code.
3. **Unconditional Statements Coverage:** The metric also encompasses the coverage of unconditional statements in the source code, such as printable statements and return statements from functions and methods.
4. **Execution of Branch Test Cases:** Branch Coverage involves the execution of test cases specifically designed for branch statements.

❖ Path Coverage

Path Testing is a method that is used to design the test cases. In the path testing method, the control flow graph of a program is designed to find a set of linearly independent paths of execution. In this method, Cyclomatic Complexity is used to determine the number of linearly independent paths and then test cases are generated for each path.

Path Testing Process



- **Control Flow Graph:**
Draw the corresponding control flow graph of the program in which all the executable paths are to be discovered.
- **Cyclomatic Complexity:**
After the generation of the control flow graph, calculate the cyclomatic complexity of the program using the following formula.
 - McCabe's Cyclomatic Complexity = $E - N + 2P$
 -
 - Where,
 - E = Number of edges in the control flow graph
 - N = Number of vertices in the control flow graph
 - P = Program factor
- **Make Set:**
Make a set of all the paths according to the control flow graph and calculate cyclomatic complexity. The cardinality of the set is equal to the calculated cyclomatic complexity.
- **Create Test Cases:**
Create a test case for each path of the set obtained in the above step.

❖ Advanced White Box Techniques

- **Path coverage**
A white box penetration testing technique that examines all possible execution paths in an application or software.
- **Automated visual testing**
An advanced white box product validation test that can be performed on software.
- **Data flow testing**
A white box testing strategy that focuses on data variables and their values.
- **Path testing**
A white box testing method that helps design test cases for all possible paths.

- **Sanity testing**

A white box testing technique that validates the basic functionality of software after major modifications.

Other white box testing techniques include: Integration testing, Performance testing, and System testing.

❖ **Intuitive and Experience Based Testing**

Experience-based testing isn't your typical testing method—it's a dynamic approach that relies on a tester's intuition, skills, and past experiences. This technique transforms these insights into concrete test scenarios, benefiting from the combined expertise of developers, testers, and users. Through collaborative efforts, this approach shapes effective tests that truly count.

Experience-based testing truly shines through its ability to uncover test scenarios that might slip through the cracks of other rigid methodologies. While structured methods have their merits, experience-based testing adds a layer of creativity and resourcefulness to the mix. This approach could be your project's game-changer to stand out in a testing landscape where thoroughness is key to success.